

RELATÓRIO — CLEAN CODE

Etapa 1 - Análise de Boas Práticas em 3 Métodos

01

validarTransicaoDeStatus

SolicitacaoService.java

QUAIS PRÁTICAS DE CLEAN CODE FORAM APLICADAS?

Nomes que revelam intenção (legibilidade) e Responsabilidade Única (SRP).

COMO FORAM APLICADAS — O QUE MUDOU NO CÓDIGO?

O nome do método descreve exatamente o que ele faz sem necessidade de comentário adicional. Os parâmetros atual e novo deixam claro o papel de cada valor. Internamente, um switch centralizado lista todas as transições válidas (ABERTO → TRIAGEM → AVALIAÇÃO → ENCERRADO) em um único lugar, sem lógica espalhada pelo serviço.

POR QUE ISSO MELHORA A MANUTENÇÃO?

Para adicionar ou alterar uma transição basta editar o switch neste único método. Não há risco de lógica duplicada esquecida em outro ponto do código. As mensagens de erro padronizadas também reduzem o tempo de diagnóstico em produção.

02

validarAtualizacaoStatus

HistoricoStatusService.java

QUAIS PRÁTICAS DE CLEAN CODE FORAM APLICADAS?

Fail Fast (captura precoce de erros), DRY (validação centralizada) e nomes descritivos nas mensagens de erro.

COMO FORAM APLICADAS — O QUE MUDOU NO CÓDIGO?

Todas as verificações de dados estão no início do método: DTO nulo, comentário vazio, ID da solicitação inexistente e ausência do gestor. Se qualquer regra falhar, o método lança exceção imediatamente, sem tentar continuar. Antes dessa centralização, cada método que chamava o serviço repetiria as mesmas verificações de forma independente.

POR QUE ISSO MELHORA A MANUTENÇÃO?

Quando um novo campo obrigatório surgir, a regra é adicionada aqui uma única vez e se propaga automaticamente para todos os fluxos. Elimina o risco de esquecer uma validação em algum ponto isolado, garantindo consistência nos dados gravados.

03

imprimirDetalhes

AcompanhamentoUI.java

QUAIS PRÁTICAS DE CLEAN CODE FORAM APLICADAS?

DRY (eliminação de duplicação) e Separação de Responsabilidades (separar busca de dados da apresentação).

COMO FORAM APLICADAS — O QUE MUDOU NO CÓDIGO?

Em vez de copiar o mesmo bloco de `System.out.println` duas vezes uma no menu do cidadão e outra no menu do gestor, criou-se um único método reutilizável. Ele cuida exclusivamente da formatação e exibição na tela. A busca dos dados no banco fica em outro método separado.

POR QUE ISSO MELHORA A MANUTENÇÃO?

Qualquer alteração no layout (novo campo, mudança de formato) é feita em um único lugar e reflete automaticamente nas duas interfaces. Elimina o problema clássico de corrigir em um menu e esquecer no outro, mantendo a experiência de cidadão e gestor sempre consistente.